

Claude Code 毎日レッスン完全カリキュラム

～ "Build Your Own X" 全カテゴリ網羅 × Claude Code 実践マスター ～

最終目標: Claude Codeでコードを書いてもらい、そのコードの修正・エラー直しを自分で行い、実装の速度と精度を上げていく

カリキュラム全体像

フェーズ	期間	テーマ	対応カテゴリ
Phase 0	Day 1-7	環境構築 & Claude Code 基礎	—
Phase 1	Day 8-28	コア基盤システム	Shell, Regex, Template Engine, CLI Tool
Phase 2	Day 29-56	Web & サーバー技術	Web Server, Database, Search Engine, Web Browser
Phase 3	Day 57-84	言語・コンパイラ・Git内 部	Programming Language, Git, Text Editor
Phase 4	Day 85-112	ネットワーク & インフラ	Network Stack, Docker, BitTorrent Client, Bot
Phase 5	Day 113-140	フロントエンド & ゲーム	Front-end Framework, Game, Physics Engine, Voxel Engine
Phase 6	Day 141-168	AI/ML & グラフィクス	Neural Network, Visual Recognition, 3D Renderer, AR
Phase 7	Day 169-189	応用・統合	Blockchain, Emulator/VM, OS, Uncategorized
Phase 8	Day 190-200	総合実践 & 卒業プロジェクト	全カテゴリ統合

毎日のレッスン進め方（共通フロー）

1. 【予習 5分】 今日の参考チュートリアルを軽く眺める
2. 【Claude Code生成 10分】 VSCodeでClaude Codeにコード生成を依頼
3. 【読解 15分】 生成されたコードを読み、構造を理解する
4. 【意図的バグ修正 20分】 Claude Codeが出したエラーや改善点を自分で修正
5. 【動作確認 5分】 修正後のコードを実行・テスト
6. 【振り返り 5分】 学んだことをコメントで記録

VSCode上での操作: すべてのレッスンはVSCode + Claude Code拡張で実施

Phase 0: 環境構築 & Claude Code 基礎（Day 1-7）

Day 1: VSCode + Claude Code セットアップ

目標: 開発環境を完全に整える

やること:

- VSCodeインストール・日本語化
- Claude Code拡張機能のインストールと認証設定
- Python 3.12 / Node.js 20 LTS インストール確認
- Git初期設定 (`git config --global`)

Claude Codeへの依頼例:

VSCodeのsettings.jsonを作成して。Python開発に最適な設定で、フォーマッター(black)、リンター(ruff)、タブサイズ4を含めて。

自分で修正する練習:

- 生成されたsettings.jsonのパスが自分の環境と合っているか確認・修正
 - 拡張機能IDが最新か確認
-

Day 2: Claude Code 基本コマンドと対話パターン

目標: Claude Codeへの効果的な指示の出し方を覚える

やること:

- 簡単なPythonスクリプト生成を依頼

- エラーが出たらClaude Codeに聞く前に自分で調べる練習
- プロンプトの書き方パターン（具体的 vs 抽象的）

Claude Codeへの依頼例:

Pythonで「Hello World」を10回表示するスクリプトを書いて。
ただし、3の倍数の時は「Fizz」、5の倍数の時は「Buzz」に変えて。

自分で修正する練習:

- 出力フォーマットを自分好みに変更
 - エッジケース（15の倍数）の処理を確認・修正
-

Day 3: プロジェクト構成の基本

目標: 標準的なプロジェクトディレクトリ構成を理解する

Claude Codeへの依頼例:

Pythonプロジェクトの標準的なディレクトリ構成を作って。
src/, tests/, docs/ を含み、setup.py, requirements.txt,
README.md, .gitignore も含めて。

自分で修正する練習:

- `.gitignore`に自分の環境固有の除外パターンを追加
 - `requirements.txt`のバージョン指定を適切に修正
-

Day 4: Git操作×Claude Code

目標: Claude Codeを使いながらGitワークフローを実践

やること:

- リポジトリ初期化、ブランチ作成、コミット、マージ
- Claude Codeに`.gitignore`生成を依頼
- コンフリクト解消の練習

Claude Codeへの依頼例:

Gitのブランチ戦略を説明するREADME.mdを書いて。
feature branch, develop, mainの3ブランチ構成で。
図はMermaidで。

Day 5: テスト駆動開発（TDD）入門

目標: pytest を使ったテストの書き方を覚える

Claude Codeへの依頼例:

電卓クラス（Calculator）をPythonで作って。
add, subtract, multiply, divideメソッドを持ち、
それぞれのpytestテストも書いて。

自分で修正する練習:

- ゼロ除算のテストケースを追加
- 浮動小数点の丸め誤差テストを追加

Day 6: デバッグ技法の基礎

目標: VSCodeデバッガとprint/loggingデバッグを使いこなす

Claude Codeへの依頼例:

意図的にバグを3つ含んだPythonスクリプトを書いて。
バグの種類: IndexError, TypeError, 論理エラーを1つずつ。
ただしバグの場所は教えないで。

自分で修正する練習:

- VSCodeのブレイクポイントを使ってバグを特定
- `launch.json`のデバッグ設定を自分で書く

Day 7: Phase 0 総復習 & ミニプロジェクト

目標: ここまでの知識を統合する

Claude Codeへの依頼例:

TODOリスト管理のCLIツールをPythonで作って。

- タスクの追加、一覧、完了、削除ができる
- JSONファイルにデータ保存
- pytestテスト付き
- argparseでコマンドライン引数进行处理

自分で修正する練習:

- 日本語対応（エンコーディング）
- タスクの優先度機能を自分で追加

Phase 1: コア基盤システム（Day 8-28）

Build Your Own Shell（Day 8-14）

参考チュートリアル:

- [C: Tutorial - Write a Shell in C](#)
- [C: Let's build a shell!](#)
- [C: Writing a UNIX Shell](#)
- [C: Build Your Own Shell](#)
- [C: Write a shell in C](#)
- [Go: Writing a simple shell in Go](#)
- [Rust: Build Your Own Shell using Rust](#)

Day 8: Shellの仕組みを理解する

Claude Codeへの依頼:

Pythonで最小限のシェル（REPL）を作って。

ユーザーからコマンドを受け取り、subprocessで実行して結果を表示する。

プロンプトは「myshell>」で。

自分で修正: コマンドが見つからない時のエラーハンドリングを追加

Day 9: パイプとリダイレクト

Claude Codeへの依頼:

Day8のシェルにパイプ (|) 機能を追加して。

例: `ls | grep .py` が動くように。

自分で修正: リダイレクト (`>`, `>>`) を自分で実装

Day 10: 組み込みコマンド (`cd`, `exit`, `help`)

Claude Codeへの依頼:

シェルに組み込みコマンドを追加して。

`cd` (ディレクトリ移動)、`exit` (終了)、`help` (ヘルプ表示)、
`pwd` (現在ディレクトリ表示) を実装。

自分で修正: `history` コマンド (コマンド履歴表示) を自分で追加

Day 11: 環境変数とエクスポート

Claude Codeへの依頼:

シェルに環境変数機能を追加。

`export KEY=VALUE` で設定、`echo $KEY` で参照、
`env` で一覧表示ができるように。

自分で修正: 変数展開の不具合 (スペースを含む値など) を修正

Day 12: ジョブ制御 (バックグラウンド実行)

Claude Codeへの依頼:

コマンド末尾に `&` を付けるとバックグラウンド実行する機能を追加。

`jobs` コマンドで実行中のジョブ一覧を表示。

自分で修正: Ctrl+Cでフォアグラウンドプロセスだけ停止するよう修正

Day 13: タブ補完とコマンド履歴

Claude Codeへの依頼:

`readline` ライブラリを使って、タブキーでファイル名補完、

上下矢印キーでコマンド履歴を辿れるようにして。

履歴は `.myshell_history` ファイルに永続化。

自分で修正: 補完候補が複数ある場合の表示を改善

Day 14: Shell総合テスト & リファクタリング

Claude Codeへの依頼:

これまで作ったシェルの全機能のテストスイートをpytestで書いて。
また、コード全体をクラスベースにリファクタリングして。

自分で修正: エッジケースのテスト追加、ドキュメント整備

📁 Build Your Own Regex Engine (Day 15-19)

参考チュートリアル:

- [C: A Regular Expression Matcher](#)
- [C: Regular Expression Matching Can Be Simple And Fast](#)
- [Go: How to build a regex engine from scratch](#)
- [JavaScript: Build a Regex Engine in Less than 40 Lines of Code](#)
- [JavaScript: How to implement regular expressions in functional javascript using derivatives](#)
- [JavaScript: Implementing a Regular Expression Engine](#)
- [Perl: How Regexes Work](#)
- [Python: Build Your Own Regular Expression Engines: Backtracking, NFA, DFA](#)
- [Scala: No Magic: Regular Expressions](#)

Day 15: リテラルマッチング

Claude Codeへの依頼:

Pythonで最小限の正規表現エンジンを作って。
まずはリテラル文字列のマッチングだけ。
`match("hello", "hello world") -> True`

自分で修正: 部分マッチと完全マッチの区別を追加

Day 16: メタ文字 (`.` と `*`)

Claude Codeへの依頼:

正規表現エンジンに `.` (任意の1文字) と `*` (0回以上繰り返し) を追加。
`match("h.llo", "hello") -> True`
`match("he*lo", "heelo") -> True`

自分で修正: `+` (1回以上) と `?` (0回か1回) を自分で追加

Day 17: 文字クラスとアンカー

Claude Codeへの依頼:

[abc] 形式の文字クラス、^ (先頭)、\$ (末尾) アンカーを実装。

自分で修正: `[a-z]` のような範囲指定を自分で実装

Day 18: NFA (非決定性有限オートマトン) による実装

Claude Codeへの依頼:

NFAベースの正規表現エンジンをPythonで実装して。
Thompson's constructionアルゴリズムを使って。
状態遷移図を可視化する関数も付けて。

自分で修正: グループ化 `()` の実装を追加

Day 19: Regexエンジンの性能テスト

Claude Codeへの依頼:

作った正規表現エンジンとPythonのreモジュールの性能比較を行う
ベンチマークスクリプトを作って。悪意のあるパターン (ReDoS)
のテストも含めて。

自分で修正: バックトラッキングの最適化を試みる

Build Your Own Template Engine (Day 20-23)

参考チュートリアル:

- [JavaScript: JavaScript template engine in just 20 lines](#)
- [JavaScript: Understanding JavaScript Micro-Templating](#)
- [Python: Approach: Building a toy template engine in Python](#)
- [Python: A Template Engine](#)
- [Ruby: How to write a template engine in less than 30 lines of code](#)

Day 20: 変数置換の基本

Claude Codeへの依頼:


```
Pythonでテンプレートエンジンを作って。  
{{ variable }} を実際の値に置換する機能。  
render("Hello {{ name }}!", {"name": "World"}) -> "Hello World!"
```

自分で修正: ネストされた辞書 (`{{ user.name }}`) 対応を追加

Day 21: 制御構文 (if/for)

Claude Codeへの依頼:

```
テンプレートエンジンに制御構文を追加。  
{% if condition %} ... {% endif %}  
{% for item in items %} ... {% endfor %}
```

自分で修正: `{% elif %}` と `{% else %}` を自分で追加

Day 22: フィルターとテンプレート継承

Claude Codeへの依頼:

```
パイプ形式のフィルター {{ name | upper }} と、  
テンプレート継承 {% extends "base.html" %} {% block content %}  
を実装して。
```

自分で修正: カスタムフィルター登録機能を追加

Day 23: HTMLエスケープとセキュリティ

Claude Codeへの依頼:

```
テンプレートエンジンにXSS対策の自動HTMLエスケープを追加。  
{{ variable }} は自動エスケープ、  
{{ variable | safe }} はエスケープしない、を実装。  
テストケースも書いて。
```

自分で修正: CSRFトークン生成ヘルパーを追加

Build Your Own Command-Line Tool (Day 24-28)

参考チュートリアル:

- [Go: Visualize your local git contributions with Go](#)
- [Go: Build a command line app with Go: lolcat](#)

- [Go: Building a cli command with Go: cowsay](#)
- [Go: Go CLI tutorial: fortune clone](#)
- [Nim: Writing a stow alternative to manage dotfiles](#)
- [Node.js: Create a CLI tool in Javascript](#)
- [Rust: Command line apps in Rust](#)
- [Rust: Writing a Command Line Tool in Rust](#)
- [Zig: Build Your Own CLI App in Zig from Scratch](#)

Day 24: argparseマスター

Claude Codeへの依頼:

Pythonのargparseを使って、ファイル検索CLIツールを作って。
--path (検索ディレクトリ), --pattern (ファイル名パターン),
--recursive (再帰検索フラグ), --size-limit (サイズ上限)
のオプションを持つ。

自分で修正: `--exclude` パターンオプションを追加

Day 25: プログレスバーと色付き出力

Claude Codeへの依頼:

CLIツールにプログレスバー (tqdmまたは自作) と
色付き出力 (ANSIエスケープシーケンス) を追加。
lolcat風の虹色テキスト出力関数も作って。

自分で修正: Windows環境での色表示対応を追加

Day 26: 設定ファイルとインタラクティブモード

Claude Codeへの依頼:

CLIツールにYAML設定ファイルのサポートと、
対話型設定ウィザード (inquirerライク) を追加。
~/.mytool/config.yml に保存。

自分で修正: 設定のバリデーションとデフォルト値処理を改善

Day 27: CLIツールのパッケージング

Claude Codeへの依頼:

作ったCLIツールをpipでインストール可能にする
pyproject.toml / setup.cfg を作って。
console_scripts エントリーポイントも設定。

自分で修正: README.md のインストール手順を整備

Day 28: Phase 1 総合プロジェクト

Claude Codeへの依頼:

Phase 1の知識を統合して、「ミニGrep」ツールを作って。

- 正規表現検索（自作エンジン使用可）
- ファイル/ディレクトリの再帰検索
- マッチ行のハイライト表示
- --count, --line-number, --ignore-case オプション
- 検索結果のテンプレートによるフォーマット出力

Phase 2: Web & サーバー技術（Day 29-56）

Build Your Own Web Server（Day 29-38）

参考チュートリアル:

- [C#: Writing a Web Server from Scratch](#)
- [Node.js: Build Your Own Web Server From Scratch In JavaScript](#)
- [Node.js: Let's code a web server from scratch with NodeJS Streams](#)
- [Node.js: lets-build-express](#)
- [PHP: Writing a webserver in pure PHP](#)
- [Python: A Simple Web Server](#)
- [Python: Let's Build A Web Server.](#)
- [Python: Web application from scratch](#)
- [Python: Building a basic HTTP Server from scratch in Python](#)
- [Python: Implementing a RESTful Web API with Python & Flask](#)
- [Ruby: Building a simple websockets server from scratch in Ruby](#)

Day 29: TCPソケット通信の基本

Claude Codeへの依頼:

Pythonのsocketモジュールだけで、TCP接続を受け付けて
"Hello, World!" を返すサーバーを作って。ポート8080で。

自分で修正: 複数クライアント同時接続（スレッド使用）に対応

Day 30: HTTPリクエストパーサー

Claude Codeへの依頼:

HTTP/1.1リクエストを解析するパーサーをPythonで作って。
メソッド、パス、ヘッダー、ボディを辞書に変換する。

自分で修正: 不正なリクエストに対する400エラー応答を追加

Day 31: HTTPレスポンスビルダー

Claude Codeへの依頼:

HTTPレスポンスを構築するクラスを作って。
ステータスコード、ヘッダー、ボディを設定して
正しいHTTPレスポンス文字列を生成する。
Content-Type, Content-Length は自動計算。

自分で修正: CORS ヘッダーの追加、圧縮（gzip）対応

Day 32: 静的ファイルサーバー

Claude Codeへの依頼:

指定ディレクトリの静的ファイルを配信するHTTPサーバーを作って。
MIMEタイプの自動判定、404ページ、ディレクトリインデックス表示。

自分で修正: パストラバーサル攻撃の防止を追加

Day 33: ルーティングシステム

Claude Codeへの依頼:

ExpressライクなURLルーティングシステムを作って。
@app.route("/users/<int:id>") のようなデコレータで
パスパラメータを処理できるように。

自分で修正: 正規表現パターンルートをサポートを追加

Day 34: ミドルウェアパイプライン

Claude Codeへの依頼:

ミドルウェアのチェーン実行機能を実装。
ログ記録、認証チェック、リクエスト変換などの
ミドルウェアを順番に実行するパイプライン。

自分で修正: エラーハンドリングミドルウェアを追加

Day 35: JSONレスポンスとREST API

Claude Codeへの依頼:

RESTful APIエンドポイントを作るフレームワークにして。
GET/POST/PUT/DELETE 対応、JSONリクエスト/レスポンス、
入力バリデーション機能付き。

自分で修正: ページネーション機能を追加

Day 36: WebSocket対応

Claude Codeへの依頼:

WebSocketのハンドシェイクと双方向通信を実装して。
簡単なチャットルーム機能を作る。

自分で修正: 複数ルームのサポートを追加

Day 37: 非同期I/O (asyncio) 対応

Claude Codeへの依頼:

サーバーをasyncioベースに書き換えて。
同時に数百のコネクションを処理できるように。
パフォーマンス比較のベンチマークも。

自分で修正: graceful shutdownの実装

Day 38: Webサーバー総合テスト

Claude Codeへの依頼:

作ったWebサーバーの統合テストスイートを作って。
各エンドポイント、エラーケース、WebSocket、
負荷テスト (locust使用) を含む。

Build Your Own Database (Day 39-48)

参考チュートリアル:

- [C: Let's Build a Simple Database](#)
- [C++: Build Your Own Redis from Scratch](#)
- [C#: Build Your Own Database](#)
- [Clojure: An Archaeology-Inspired Database](#)
- [Crystal: Why you should build your own NoSQL Database](#)
- [Go: Build Your Own Database from Scratch](#)
- [Go: Code a database in 45 steps](#)
- [Go: Build Your Own Redis from Scratch](#)
- [JavaScript: Dagoba: an in-memory graph database](#)
- [Python: DBDB: Dog Bed Database](#)
- [Python: Write your own miniature Redis with Python](#)
- [Ruby: Build your own fast, persistent KV store in Ruby](#)
- [Rust: Build your own Redis client and server](#)

Day 39: Key-Valueストアの基本

Claude Codeへの依頼:

Pythonでインメモリ Key-Value ストアを作って。
GET, SET, DELETE, EXISTS 操作。
ハッシュマップで実装し、TTL (有効期限) 機能付き。

自分で修正: LRUキャッシュの追加

Day 40: データの永続化 (ファイルストレージ)

Claude Codeへの依頼:

Key-ValueストアにWAL (Write-Ahead Log) による
永続化を追加。サーバー再起動後もデータが残るように。

自分で修正: スナップショット (定期的な全データ保存) 機能を追加

Day 41: B+Treeインデックス

Claude Codeへの依頼:

B+Tree（次数4）をPythonで実装して。
挿入、検索、削除、範囲検索をサポート。
ツリーの状態を可視化する関数も。

自分で修正: ディスクベースのB+Tree（ページ管理）に拡張

Day 42: SQLパーサー（基本）

Claude Codeへの依頼:

簡易SQLパーサーを作って。
SELECT, INSERT, UPDATE, DELETE, CREATE TABLE,
WHERE句をパースしてAST（抽象構文木）に変換。

自分で修正: ORDER BY, LIMIT, JOIN の解析を追加

Day 43: クエリエンジン

Claude Codeへの依頼:

パースしたSQLを実際に行うクエリエンジンを作って。
テーブルはCSVファイルで管理。
WHERE句のフィルタリング、SELECT句の射影を処理。

自分で修正: GROUP BY と集約関数（COUNT, SUM, AVG）を追加

Day 44: トランザクション（ACID）

Claude Codeへの依頼:

データベースにトランザクション機能を追加。
BEGIN, COMMIT, ROLLBACK を実装。
WALを使ったクラッシュリカバリも。

自分で修正: ロック機構（排他制御）を追加

Day 45: Redisクローン（RESPプロトコル）

Claude Codeへの依頼:

Redis互換のRESPプロトコルパーサーとサーバーを作って。
PING, SET, GET, DEL, INCR, LPUSH, LRANGE コマンドに対応。
redis-cli からアクセスできるように。

自分で修正: PUB/SUB機能を追加

Day 46: グラフデータベース

Claude Codeへの依頼:

シンプルなインメモリグラフデータベースを作って。
ノードとエッジの追加、最短経路探索（BFS/Dijkstra）、
パターンマッチクエリ。

自分で修正: ノードのプロパティ検索とインデックスを追加

Day 47: データベースのREPLインターフェース

Claude Codeへの依頼:

作ったデータベースにインタラクティブなSQLコンソール（REPL）
を追加。テーブル表示の整形、コマンド履歴、
.help, .tables, .schema などのメタコマンド対応。

Day 48: Phase 2前半まとめ — DB性能テスト

Claude Codeへの依頼:

データベースのベンチマークスイートを作って。
挿入/検索/更新/削除の各操作で1万件~10万件の
パフォーマンスを計測してグラフ化。

Build Your Own Search Engine (Day 49-53)

参考チュートリアル:

- [CSS: A search engine in CSS](#)
- [Python: Building a search engine using Redis and redis-py](#)
- [Python: Building a Vector Space Indexing Engine in Python](#)
- [Python: Building A Python-Based Search Engine \[video\]](#)
- [Python: Making text search learn from feedback](#)
- [Python: Finding Important Words in Text Using TF-IDF](#)

Day 49: 転置インデックスの構築

Claude Codeへの依頼:

Pythonで転置インデックス (Inverted Index) を作って。
テキストファイル群を読み込み、単語→文書IDリストのマッピングを構築。
トークナイズ、ストップワード除去、ステミング付き。

自分で修正: 日本語形態素解析 (MeCab/janome) 対応を追加

Day 50: TF-IDF ランキング

Claude Codeへの依頼:

TF-IDF (Term Frequency - Inverse Document Frequency)
スコアリングを実装して検索結果をランキング。

自分で修正: BM25 ランキングアルゴリズムに改良

Day 51: ブーリアン検索とフレーズ検索

Claude Codeへの依頼:

AND, OR, NOT のブーリアン演算と、
"exact phrase" のフレーズ検索を実装。

自分で修正: ワイルドカード検索 (`(prog*)`) を追加

Day 52: ベクトル空間モデル

Claude Codeへの依頼:

文書と検索クエリをベクトルに変換し、
コサイン類似度で検索結果をランキングする
ベクトル空間検索エンジンを作って。

自分で修正: Word2Vec埋め込みによるセマンティック検索に拡張

Day 53: 検索エンジンWeb UI

Claude Codeへの依頼:

検索エンジンにFlaskベースのWeb UIを追加。
検索ボックス、結果一覧 (スニペット付き)、
ページネーション、検索にかかった時間の表示。

Build Your Own Web Browser (Day 54-56)

参考チュートリアル:

- [Rust: Let's build a browser engine](#)
- [Python: Browser Engineering](#)

Day 54: HTMLパーサー

Claude Codeへの依頼:

簡易HTMLパーサーをPythonで作って。
HTML文字列をDOMツリー（Nodeオブジェクト）に変換。
タグ、属性、テキストノードを正しく解析。

自分で修正: 自己閉じタグ（`<br/>`等）と属性値のクォート処理を修正

Day 55: CSSパーサーとスタイル計算

Claude Codeへの依頼:

CSSセクタパーサーとスタイル規則を実装。
タグ名、クラス、IDセクタに対応。
DOMツリーの各ノードに計算済みスタイルを適用。

自分で修正: セクタの詳細度（specificity）計算を修正

Day 56: レイアウトエンジンとレンダリング

Claude Codeへの依頼:

ボックスモデルに基づくレイアウトエンジンを作って。
ブロック要素のレイアウト計算を行い、
Pillowライブラリで結果を画像としてレンダリング。

自分で修正: テキストの折り返しとインライン要素の配置を改善

Phase 3: 言語・コンパイラ・Git内部 (Day 57-84)

Build Your Own Programming Language (Day 57-72)

参考チュートリアル:

- [\(any\): mal - Make a Lisp](#)
- [C: Baby's First Garbage Collector](#)

- [C: Build Your Own Lisp](#)
- [C: Writing a Simple Garbage Collector in C](#)
- [C: C interpreter that interprets itself](#)
- [C: A journey explaining how to build a compiler from scratch](#)
- [C++: Writing Your Own Toy Compiler Using Flex](#)
- [C++: How to Create a Compiler \[video\]](#)
- [C++: Kaleidoscope: Implementing a Language with LLVM](#)
- [F#: Understanding Parser Combinators](#)
- [Elixir: Demystifying compilers by writing your own \[video\]](#)
- [Go: The Super Tiny Compiler](#)
- [Go: Lexical Scanning in Go \[video\]](#)
- [Haskell: Let's Build a Compiler](#)
- [Haskell: Write You a Haskell](#)
- [Haskell: Write Yourself a Scheme in 48 Hours](#)
- [Haskell: Write You A Scheme](#)
- [Java: Crafting interpreters](#)
- [Java: Creating JVM Language](#)
- [JavaScript: The Super Tiny Compiler](#)
- [JavaScript: The Super Tiny Interpreter](#)
- [JavaScript: Little Lisp interpreter](#)
- [JavaScript: How to implement a programming language in JavaScript](#)
- [JavaScript: Let's go write a Lisp](#)
- [OCaml: Writing a C Compiler](#)
- [OCaml: Writing a Lisp, the series](#)
- [Pascal: Let's Build a Compiler](#)
- [Python: A Python Interpreter Written in Python](#)
- [Python: lisp.py: Make your own Lisp interpreter](#)
- [Python: How to Write a Lisp Interpreter in Python](#)
- [Python: Let's Build A Simple Interpreter](#)
- [Python: Make Your Own Simple Interpreted Programming Language \[video\]](#)
- [Python: From Source Code To Machine Code](#)
- [Racket: Beautiful Racket](#)
- [Ruby: A Compiler From Scratch](#)

- [Ruby: Markdown compiler from scratch in Ruby](#)
- [Rust: Learning Parser Combinators With Rust](#)
- [Swift: Building a LISP from scratch with Swift](#)
- [TypeScript: Build your own WebAssembly Compiler](#)

Day 57: レキサー（字句解析器）

Claude Codeへの依頼:

```
Pythonで字句解析器（Lexer/Tokenizer）を作って。  
対応するトークン：数値、文字列、識別子、演算子（+, -, *, /, ==, !=）、  
括弧、キーワード（if, else, while, def, return, print）。
```

自分で修正: コメント（`# ...`）と複数行文字列のサポートを追加

Day 58: パーサー（構文解析器）－式の解析

Claude Codeへの依頼:

```
再帰下降パーサーで算術式をAST（抽象構文木）に変換。  
演算子の優先順位を正しく処理（掛け算 > 足し算）。  
ASTをJSON形式でダンプする関数も。
```

自分で修正: 単項演算子（`-x`）と括弧のネストを修正

Day 59: パーサー－文と制御構文

Claude Codeへの依頼:

```
パーサーに文レベルの構文を追加。  
- 変数代入: x = 10  
- if/else文  
- whileループ  
- 関数定義: def func(a, b): ...  
- return文
```

自分で修正: `for i in range(n)` ライクな構文を追加

Day 60: ASTインタプリタ（基本）

Claude Codeへの依頼:

ASTを巡回して実行するインタプリタを作って。
変数の環境（スコープ）管理、算術演算、比較、
print文の実行。

自分で修正: 変数のスコープ（ローカル/グローバル）を正しく処理

Day 61: 関数とクロージャ

Claude Codeへの依頼:

インタプリタに関数呼び出しとクロージャを実装。
再帰呼び出し（フィボナッチ等）が動くように。

自分で修正: デフォルト引数と可変長引数のサポートを追加

Day 62: データ構造（リストと辞書）

Claude Codeへの依頼:

言語にリスト [1, 2, 3] と辞書 {"a": 1} を追加。
インデックスアクセス、len()、append()、keys() メソッド対応。

自分で修正: スライス (`list[1:3]`) 操作を追加

Day 63: エラーハンドリング

Claude Codeへの依頼:

try/except/finally 構文をサポート。
カスタム例外クラスとスタックトレース表示。
構文エラー時の行番号・列番号表示も。

自分で修正: エラーメッセージの日本語化

Day 64: 標準ライブラリ

Claude Codeへの依頼:

組み込み関数セットを追加。
print, len, range, type, str, int, float, input,
append, pop, keys, values。

自分で修正: ファイルI/O (open, read, write) を追加

Day 65: REPL（対話型環境）

Claude Codeへの依頼:

言語の対話型REPL環境を作って。
複数行入力、シンタックスハイライト（Pygments使用）、
タブ補完、コマンド履歴対応。

Day 66: Lispインタプリタ

Claude Codeへの依頼:

Peter NorvigのLispy (How to Write a Lisp Interpreter in Python)
に基づいて、Schemeサブセットのインタプリタを作って。
S式パーサー、環境、eval/applyサイクル。

自分で修正: マクロシステム（defmacro）を追加

Day 67: バイトコードコンパイラ

Claude Codeへの依頼:

ASTをバイトコード（中間表現）にコンパイルするコンパイラを作って。
LOAD_CONST, STORE_VAR, BINARY_ADD, CALL_FUNCTION,
JUMP_IF_FALSE 等の命令セット。

自分で修正: 最適化パス（定数量み込み）を追加

Day 68: バイトコードVM（仮想マシン）

Claude Codeへの依頼:

バイトコードを実行するスタックベースの仮想マシンを作って。
命令フェッチ→デコード→実行のサイクル。

自分で修正: デバッグモード（ステップ実行、スタック表示）を追加

Day 69: ガベージコレクター

Claude Codeへの依頼:

マーク&スイープ方式のガベージコレクタを実装して。
オブジェクトの参照追跡、ルート集合からのマーク、
到達不能オブジェクトの解放。

自分で修正: 世代別GCに改良

Day 70: 型チェッカー

Claude Codeへの依頼:

シンプルな静的型チェッカーを作って。

型注釈: `x: int = 10`, `def add(a: int, b: int) -> int`

型推論と型エラーの検出。

自分で修正: ジェネリクス型のサポートを追加

Day 71: Markdownコンパイラ

Claude Codeへの依頼:

Markdown→HTMLのコンパイラを作って。

見出し、段落、リスト、コードブロック、リンク、画像、
太字、斜体をサポート。

自分で修正: テーブル記法とタスクリストを追加

Day 72: 言語プロジェクト総合テスト

Claude Codeへの依頼:

自作プログラミング言語で以下のプログラムが動くか検証:

- FizzBuzz
 - フィボナッチ数列（再帰）
 - クイックソート
 - 簡易HTTPクライアント
- テストスイートを作って。

Build Your Own Git (Day 73-79)

参考チュートリアル:

- [Haskell: Reimplementing "git clone" in Haskell from the bottom up](#)
- [JavaScript: Gitlet](#)
- [JavaScript: Build GIT - Learn GIT](#)
- [Python: Just enough of a Git client to create a repo, commit, and push itself to GitHub](#)
- [Python: Write yourself a Git!](#)
- [Python: ugit: Learn Git Internals by Building Git Yourself](#)
- [Ruby: Rebuilding Git in Ruby](#)

Day 73: Gitオブジェクトモデル (blob, tree)

Claude Codeへの依頼:

Gitのオブジェクトモデルを実装。
SHA-1ハッシュによるblob, treeオブジェクトの作成と保存。
.git/objects/ にzlib圧縮で保存。

自分で修正: オブジェクトの読み込みとcat-file機能を追加

Day 74: コミットオブジェクトとログ

Claude Codeへの依頼:

commitオブジェクトを作成。
作者、コミッター、メッセージ、親コミットの参照。
git log ライクなコミット履歴表示。

自分で修正: コミットグラフの可視化を追加

Day 75: init, add, commit

Claude Codeへの依頼:

mygit init (リポジトリ初期化)
mygit add <file> (ステージング)
mygit commit -m "message" (コミット作成)
を実装。インデックスファイルの管理も。

自分で修正: `.mygitignore` ファイルのサポートを追加

Day 76: ブランチとHEAD

Claude Codeへの依頼:

ブランチの作成・切り替え・一覧表示を実装。
mygit branch <name>
mygit checkout <branch>
HEADの管理 (デタッチドHEAD含む)。

自分で修正: `mygit branch -d` (ブランチ削除) を追加

Day 77: diff と status

Claude Codeへの依頼:

mygit diff (ワーキングツリーとインデックスの差分表示)
mygit status (変更ファイルの状態表示) を実装。
差分はunified diff形式で色付き表示。

自分で修正: `--staged` オプション (ステージ済みの差分) を追加

Day 78: マージ

Claude Codeへの依頼:

3-way マージアルゴリズムを実装。
fast-forward マージと再帰マージの両方。
コンフリクト発生時のマーカー挿入。

自分で修正: コンフリクト解消のインタラクティブUIを追加

Day 79: リモートとクローン (概念実装)

Claude Codeへの依頼:

ローカルディレクトリ間での clone と push を実装。
バックファイル形式の簡易版。

Build Your Own Text Editor (Day 80-84)

参考チュートリアル:

- [C: Build Your Own Text Editor](#)
- [C++: Designing a Simple Text Editor](#)
- [Python: Python Tutorial: Make Your Own Text Editor \[video\]](#)
- [Python: Create a Simple Python Text Editor!](#)
- [Ruby: Build a Collaborative Text Editor Using Rails](#)
- [Rust: Hecto: Build your own text editor in Rust](#)

Day 80: ターミナル制御とrawモード

Claude Codeへの依頼:

Pythonのcursesライブラリでターミナルエディタのベースを作って。
rawモードでのキー入力受付、画面描画、カーソル移動。

自分で修正: ウィンドウリサイズ対応を追加

Day 81: テキストバッファとファイル操作

Claude Codeへの依頼:

Gap Buffer方式でテキストバッファを実装。
ファイルの読み込み・保存、行単位の管理。

自分で修正: Undo/Redo機能を追加

Day 82: シンタックスハイライト

Claude Codeへの依頼:

Python, JavaScript, Markdownのシンタックスハイライトを
正規表現ベースで実装。キーワード、文字列、コメント、
数値をそれぞれ色付け。

自分で修正: 新しい言語のハイライト定義を追加

Day 83: 検索・置換と行番号

Claude Codeへの依頼:

インクリメンタル検索 (Ctrl+F) と
一括置換 (Ctrl+H) を実装。行番号表示も追加。

Day 84: ステータスバーとキーバインド

Claude Codeへの依頼:

ステータスバー (ファイル名、行数、変更状態、エンコーディング)
とカスタマイズ可能なキーバインド設定。

Phase 4: ネットワーク & インフラ (Day 85-112)

Build Your Own Network Stack (Day 85-91)

参考チュートリアル:

- [C: Beej's Guide to Network Programming](#)
- [C: Let's code a TCP/IP stack](#)
- [C / Python: Build your own VPN/Virtual Switch](#)

- Ruby: How to build a network stack in Ruby

Day 85: ソケットプログラミング基礎

Claude Codeへの依頼:

TCPエコーサーバーとクライアントをPythonで作って。
マルチスレッド対応で複数クライアント同時接続可能。

Day 86: UDPとDNSリゾルバ

Claude Codeへの依頼:

UDPソケットでDNSクエリを送信し、
ドメイン名⇒IPアドレスを解決するDNSリゾルバを作って。
DNSパケットの構築と解析を自前で実装。

自分で修正: DNSキャッシュ機能を追加

Day 87: HTTPクライアント

Claude Codeへの依頼:

ソケットだけでHTTP/1.1クライアントを作って。
GET/POSTリクエスト、ヘッダー処理、
チャンク転送エンコーディング、リダイレクト追従。

Day 88: ポートスキャナ

Claude Codeへの依頼:

TCP SYN/Connect方式のポートスキャナを作って。
指定ホストの指定範囲ポートをスキャン。
asyncioで並列スキャン、サービス名判定。

Day 89: チャットサーバー (TCP)

Claude Codeへの依頼:

TCPベースのマルチルームチャットサーバー。
ニックネーム設定、ルーム作成/参加、
プライベートメッセージ、ユーザー一覧表示。

Day 90: プロキシサーバー

Claude Codeへの依頼:

HTTPフォワードプロキシサーバーを作って。
クライアント→プロキシ→対象サーバーのリレー。
アクセスログ記録とフィルタリングルール。

Day 91: ネットワーク統合テスト

Claude Codeへの依頼:

ネットワークアプリの統合テストフレームワーク。
モックサーバー、パケットキャプチャ、遅延シミュレーション。

Build Your Own Docker (Day 92-97)

参考チュートリアル:

- [C: Linux containers in 500 lines of code](#)
- [Go: Build Your Own Container Using Less than 100 Lines of Go](#)
- [Go: Building a container from scratch in Go \[video\]](#)
- [Python: A workshop on Linux containers: Rebuild Docker from Scratch](#)
- [Python: A proof-of-concept imitation of Docker, written in 100% Python](#)
- [Shell: Docker implemented in around 100 lines of bash](#)

Day 92: Linuxプロセス分離の基礎

Claude Codeへの依頼:

Pythonでプロセスの分離（namespace概念）を説明するデモを作って。
chroot, プロセスグループ、リソース制限(ulimit)の実験スクリプト。

Day 93: Dockerfileパーサー

Claude Codeへの依頼:

Dockerfile形式のパーサーを作って。
FROM, RUN, COPY, WORKDIR, ENV, EXPOSE, CMD
各命令を解析してビルドステップに変換。

Day 94: イメージレイヤー管理

Claude Codeへの依頼:

Copy-on-Writeのレイヤーファイルシステムを簡易実装。
各レイヤーをtarアーカイブで管理し、
重ね合わせてファイルシステムを構築。

Day 95: コンテナランタイム（簡易版）

Claude Codeへの依頼:

chrootとsubprocessを使った簡易コンテナランタイム。
隔離された環境でコマンドを実行。

Day 96: docker-compose風YAML解析

Claude Codeへの依頼:

docker-compose.yml を解析して、
複数サービスの定義（ポート、環境変数、依存関係）
を読み取るパーサーを作って。

Day 97: コンテナネットワーキング

Claude Codeへの依頼:

コンテナ間通信のシミュレーション。
仮想ネットワーク（ブリッジ型）の概念実装。
ポートフォワーディングのロジック。

Build Your Own BitTorrent Client（Day 98-103）

参考チュートリアル:

- [C#: Building a BitTorrent client from scratch in C#](#)
- [Go: Building a BitTorrent client from the ground up in Go](#)
- [Nim: Writing a Bencode Parser](#)
- [Node.js: Write your own bittorrent client](#)
- [Python: A BitTorrent client in Python 3.5](#)

Day 98: Bencodeエンコーダ/デコーダ

Claude Codeへの依頼:

BitTorrentのBencode形式の
エンコーダとデコーダをPythonで実装して。
文字列、整数、リスト、辞書の変換。

Day 99: .torrent ファイルの解析

Claude Codeへの依頼:

.torrentファイルを読み込んでメタデータを表示。
ファイル名、サイズ、ピース数、トラッカーURL、
info_hash (SHA-1) の計算。

Day 100: トラッカー通信

Claude Codeへの依頼:

HTTPトラッカーとの通信を実装。
announceリクエストの送信、ピアリストの取得と解析。

Day 101: ピア接続とハンドシェイク

Claude Codeへの依頼:

ピアとのTCP接続確立とBitTorrentハンドシェイク。
Peer Wire Protocol のメッセージ送受信。

Day 102: ピースのダウンロード

Claude Codeへの依頼:

ピースの要求、受信、SHA-1検証、ファイル書き込み。
複数ピアからの並列ダウンロード。

Day 103: BitTorrent統合テスト

Claude Codeへの依頼:

実際のtorrentファイルを使ったダウンロードテスト。
進捗表示、速度計算、ピア管理のダッシュボード。

Build Your Own Bot (Day 104-112)

参考チュートリアル:

- [Haskell: Roll your own IRC bot](#)
- [Node.js: Creating a Simple Facebook Messenger AI Bot with API.ai in Node.js](#)
- [Node.js: How to make a responsive telegram bot](#)
- [Node.js: Create a Discord bot](#)
- [Node.js: gifbot - Building a GitHub App](#)
- [Node.js: Building A Simple AI Chatbot With Web Speech API And Node.js](#)
- [Python: How to Build Your First Slack Bot with Python](#)
- [Python: Build a Reddit Bot](#)
- [Python: How To Create a Telegram Bot Using Python](#)
- [Python: Create a Twitter Bot in Python Using Tweepy](#)
- [Python: Creating Reddit Bot with Python & PRAW \[video\]](#)
- [R: Build A Cryptocurrency Trading Bot with R](#)
- [Rust: A bot for Starcraft in Rust, C or any other language](#)

Day 104: チャットボットフレームワーク

Claude Codeへの依頼:

Pythonでチャットボットフレームワークを作って。
ルールベース（正規表現マッチ）と
コマンドベース（/help, /weather等）の両方をサポート。
プラグインシステムで機能を拡張可能に。

Day 105: Discord Bot

Claude Codeへの依頼:

discord.pyを使ったDiscord Botを作って。
挨拶、サイコロ、TODO管理、タイマー機能。
コマンドプレフィックスは「!」。

Day 106: Telegram Bot

Claude Codeへの依頼:

python-telegram-botライブラリで
Telegram Botを作って。インラインキーボード、
画像送信、会話フロー管理。

Day 107: Webスクレイピングボット

Claude Codeへの依頼:

BeautifulSoup + requestsで
Webスクレイピングボットを作って。
指定サイトの定期監視と変更通知。
robots.txtの遵守チェックも。

Day 108: AIチャットボット

Claude Codeへの依頼:

Claude APIを使ったAIチャットボットを作って。
会話履歴の管理、コンテキスト維持、
システムプロンプトのカスタマイズ。

Day 109: GitHub App

Claude Codeへの依頼:

GitHub Webhookを受け取り、
PR作成時に自動コメントを投稿するBotを作って。
Issue自動ラベリング機能も。

Day 110: Slack Bot

Claude Codeへの依頼:

Slack Bolt for Pythonを使ったSlack Botを作って。
スラッシュコマンド、モーダル表示、
メッセージアクション対応。

Day 111: ボットのスケジューリング

Claude Codeへの依頼:

APSchedulerを使ってボットに定期実行機能を追加。
毎朝の天気通知、週次レポート自動生成。

Day 112: Phase 4 総合プロジェクト

Claude Codeへの依頼:

ここまでの知識を統合した「DevOps Bot」を作って。

- GitHub WebhookでPRイベントを受信
- 自動テスト実行（コンテナ内）
- 結果をSlack/Discord通知
- デプロイコマンド対応

Phase 5: フロントエンド & ゲーム（Day 113-140）

Build Your Own Front-end Framework（Day 113-122）

参考チュートリアル:

- [JavaScript: WTF is JSX \(Let's Build a JSX Renderer\)](#)
- [JavaScript: A DIY guide to build your own React](#)
- [JavaScript: Building React From Scratch](#) [video]
- [JavaScript: Gooact: React in 160 lines of JavaScript](#)
- [JavaScript: Build Yourself a Redux](#)
- [JavaScript: Let's Write Redux!](#)
- [JavaScript: Redux: Implementing Store from Scratch](#) [video]
- [JavaScript: Build Your own Simplified AngularJS in 200 Lines](#)
- [JavaScript: How to write your own Virtual DOM](#)
- [JavaScript: Building a frontend framework, from scratch, with components](#)
- [JavaScript: Build your own React](#)
- [JavaScript: Building a Custom React Renderer](#) [video]

Day 113: Virtual DOM の基本

Claude Codeへの依頼:

JavaScriptでVirtual DOMの基本を実装して。
createElement関数でVNodeを生成、
render関数で実際のDOMに変換。

自分で修正: テキストノードとネストされた子要素の処理を改善

Day 114: Diffアルゴリズム

Claude Codeへの依頼:

新旧2つのVirtual DOMツリーを比較して
差分（パッチ）を計算するdiff関数を作って。

自分で修正: keyプロパティによるリスト要素の効率的な差分計算

Day 115: パッチ適用とDOM更新

Claude Codeへの依頼:

diffで計算したパッチを実際のDOMに適用する
patch関数を実装。最小限のDOM操作で更新。

Day 116: コンポーネントシステム

Claude Codeへの依頼:

関数コンポーネントとクラスコンポーネントの両方を
サポートするコンポーネントシステムを作って。
props と state の管理。

Day 117: 状態管理（useState風フック）

Claude Codeへの依頼:

React の useState, useEffect に相当するフックを実装。
状態変更時の自動再レンダリング。

Day 118: Reduxクローン

Claude Codeへの依頼:

createStore, dispatch, subscribe, combineReducers
を持つ状態管理ライブラリ（Redux風）を実装。
ミドルウェアシステムも。

Day 119: JSXトランスパイラ

Claude Codeへの依頼:

JSX構文を createElement() 呼び出しに変換する
簡易トランスパイラを作って。
<div className="app">Hello</div> →
createElement("div", {className: "app"}, "Hello")

Day 120: イベントシステム

Claude Codeへの依頼:

イベント委譲パターンで効率的なイベント処理を実装。
onClick, onChange, onSubmit 等のハンドラー。
合成イベントオブジェクト。

Day 121: ルーター

Claude Codeへの依頼:

SPAルーターを実装。History API を使ったパス管理、
動的ルートマッチング、ネストルート、
リダイレクト機能。

Day 122: フレームワーク統合デモ

Claude Codeへの依頼:

自作フレームワークでTODOアプリを作って。
コンポーネント分割、状態管理、ルーティング、
フォーム処理を使った実用的なアプリ。

Build Your Own Game (Day 123-133)

参考チュートリアル（Python/JavaScript中心で抜粋）：

- [JavaScript: 2D breakout game using Phaser](#)

- [JavaScript: How to Make Flappy Bird in HTML5 With Phaser](#)
- [JavaScript: Developing Games with React, Redux, and SVG](#)
- [JavaScript: How to Make Your First Roguelike](#)
- [JavaScript: Think like a programmer: How to build Snake using only JavaScript, HTML & CSS](#)
- [Python: Developing Games With PyGame](#)
- [Python: Making Games with Python & Pygame \[pdf\]](#)
- [Python: Roguelike Tutorial Revised](#)
- [Rust: Adventures in Rust: A Basic 2D Game](#)
- [Rust: Roguelike Tutorial in Rust + tcod](#)
- その他C/C++/C#/Go/Java/Lua/Ruby の多数のチュートリアルあり

Day 123: ゲームループとキャンバス描画

Claude Codeへの依頼:

HTML5 Canvas + JavaScriptで基本ゲームループを実装。
60FPS固定、デルタタイム計算、
キー入力処理、矩形の移動。

Day 124: スプライトとアニメーション

Claude Codeへの依頼:

スプライトシートの読み込みとアニメーション再生。
フレームベースのアニメーションシステム。

Day 125: 衝突検出

Claude Codeへの依頼:

AABB（軸平行境界ボックス）と円形の衝突検出。
反射角の計算とバウンド処理。

Day 126: Snakeゲーム

Claude Codeへの依頼:

Canvas + JSでSnakeゲームを完成させて。
スコア表示、難易度上昇、ハイスコア保存。

Day 127: ブロック崩し

Claude Codeへの依頼:

Breakoutゲーム。パドル操作、ブロック配置、
ボール反射、パワーアップアイテム。

Day 128: Pygameでのゲーム開発

Claude Codeへの依頼:

Pygameでシンプルな横スクロールプラットフォーマー。
重力、ジャンプ、足場、コイン収集。

Day 129: タイルマップエンジン

Claude Codeへの依頼:

JSONベースのタイルマップエンジン。
レイヤー管理、カメラスクロール、
タイルベースの衝突判定。

Day 130: ログライクゲーム（マップ生成）

Claude Codeへの依頼:

手続き的ダンジョン生成アルゴリズム。
BSP (Binary Space Partitioning) で部屋と通路を生成。
FOV (視野) 計算。

Day 131: ログライク（戦闘とAI）

Claude Codeへの依頼:

ターンベース戦闘システム。
敵AIのパスファインディング（A*アルゴリズム）。
インベントリとアイテムシステム。

Day 132: テトリス

Claude Codeへの依頼:

テトリスの完全実装。7種テトリミノ、
回転（SRS: Super Rotation System）、
ライン消去、スコアリング、ゴースト表示。

Day 133: チェスエンジン（基本）

Claude Codeへの依頼:

チェスの盤面表現、合法手生成、
ミニマックス法（ $\alpha\beta$ 枝刈り）による思考エンジン。
ASCII表示の対局インターフェース。

Build Your Own Physics Engine（Day 134-137）

参考チュートリアル:

- [C: Video Game Physics Tutorial](#)
- [C++: Game physics series by Allen Chou](#)
- [C++: How to Create a Custom Physics Engine](#)
- [C++: 3D Physics Engine Tutorial \[video\]](#)
- [JavaScript: How Physics Engines Work](#)
- [JavaScript: Broad Phase Collision Detection Using Spatial Partitioning](#)
- [JavaScript: Build a simple 2D physics engine for JavaScript games](#)

Day 134: 剛体シミュレーション基礎

Claude Codeへの依頼:

2D剛体物理シミュレーション。
位置、速度、加速度の更新（Verlet積分）。
重力と摩擦の適用。Canvas描画。

Day 135: 衝突応答とインパルス

Claude Codeへの依頼:

SAT（Separating Axis Theorem）による衝突検出。
衝突応答（反発係数、インパルス計算）。

Day 136: 拘束とジョイント

Claude Codeへの依頼:

距離拘束、ピンジョイント、スプリングを実装。
振り子、チェーン、ソフトボディのデモ。

Day 137: 物理エンジン統合デモ

Claude Codeへの依頼:

物理エンジンを使ったAngry Birds風のゲーム。
射出角度と力の調整、建物の崩壊シミュレーション。

Build Your Own Voxel Engine (Day 138-140)

参考チュートリアル:

- [C++: Let's Make a Voxel Engine](#)

Day 138: ボクセルデータ構造

Claude Codeへの依頼:

3Dボクセル世界のデータ構造（チャンク、ブロック型）。
Octreeによる空間分割。メモリ効率の良い格納。
Pythonでデータ生成、Three.jsで可視化。

Day 139: 地形生成（Perlinノイズ）

Claude Codeへの依頼:

Perlinノイズを使った地形生成。
高さマップ、バイオーム、洞窟生成。

Day 140: メッシュ生成と描画

Claude Codeへの依頼:

可視面だけのメッシュ生成（Greedy Meshing）。
Three.jsでWebGL描画。カメラ操作。

Phase 6: AI/ML & グラフィクス (Day 141-168)

Build Your Own Neural Network (Day 141-152)

参考チュートリアル:

- [C#: Neural Network OCR](#)
- [F#: Building Neural Networks in F#](#)
- [Go: Build a multilayer perceptron with Golang](#)
- [Go: How to build a simple artificial neural network with Go](#)
- [Go: Building a Neural Net from Scratch in Go](#)
- [JavaScript / Java: Neural Networks - The Nature of Code \[video\]](#)
- [JavaScript: Neural networks from scratch for JavaScript linguists](#)
- [Python: A Neural Network in 11 lines of Python](#)
- [Python: Implement a Neural Network from Scratch](#)
- [Python: Optical Character Recognition \(OCR\)](#)
- [Python: Traffic signs classification with a convolutional network](#)
- [Python: Generate Music using LSTM Neural Network in Keras](#)
- [Python: An Introduction to Convolutional Neural Networks](#)
- [Python: Neural Networks: Zero to Hero](#)

Day 141: パーセプトロン

Claude Codeへの依頼:

単純パーセプトロンをNumPyで実装。
AND, OR, XOR ゲートの学習。
決定境界の可視化 (matplotlib)。

自分で修正: 学習率のスケジューリングを追加

Day 142: 多層パーセプトロン (順伝播)

Claude Codeへの依頼:

3層ニューラルネットワークの順伝播を実装。
入力層→隠れ層→出力層。
活性化関数: sigmoid, ReLU, tanh。

Day 143: 逆伝播法 (バックプロパゲーション)

Claude Codeへの依頼:

逆伝播法による勾配計算と重み更新を実装。
連鎖律の適用、勾配降下法。
損失関数: MSE, Cross-Entropy。

自分で修正: 勾配消失問題の確認と対策 (重み初期化の改善)

Day 144: MNISTの手書き数字認識

Claude Codeへの依頼:

自作ニューラルネットワークでMNIST手書き数字を分類。
784→128→64→10 の構成。
精度計算と混同行列の表示。

自分で修正: バッチ処理とミニバッチ勾配降下法を追加

Day 145: 最適化アルゴリズム

Claude Codeへの依頼:

SGD, Momentum, RMSProp, Adam の各最適化アルゴリズムを実装。
同じデータで収束速度を比較するグラフ。

Day 146: 正則化テクニック

Claude Codeへの依頼:

L1/L2正則化、ドロップアウト、
バッチ正規化を実装。
過学習の防止効果を可視化。

Day 147: 畳み込みニューラルネットワーク (CNN)

Claude Codeへの依頼:

畳み込み層、プーリング層をNumPyで実装。
フィルタの学習と特徴マップの可視化。

Day 148: RNNとLSTM

Claude Codeへの依頼:

Recurrent Neural Network (RNN) と
Long Short-Term Memory (LSTM) をNumPyで実装。
テキスト生成 (文字レベル言語モデル) に適用。

Day 149: 自動微分エンジン

Claude Codeへの依頼:

PyTorch のautograd風の自動微分エンジンを作って。
計算グラフの構築と逆伝播の自動実行。
Tensorクラスに演算子オーバーロード。

自分で修正: 行列演算 (matmul) のサポートを追加

Day 150: GAN (敵対的生成ネットワーク)

Claude Codeへの依頼:

シンプルなGAN (Generator + Discriminator) を実装。
2D正規分布のデータ生成で学習。
生成データの分布をアニメーション表示。

Day 151: 転移学習とファインチューニング

Claude Codeへの依頼:

事前学習済みモデル (TensorFlow/PyTorch) を読み込んで
新しいタスクにファインチューニング。
花の分類タスクでデモ。

Day 152: NN総合プロジェクト - OCR

Claude Codeへの依頼:

自作CNNで手書き日本語 (ひらがな) 認識システム。
データ前処理、モデル学習、推論API、
Webインターフェースまで一気通貫。

Build Your Own Visual Recognition System (Day 153-156)

参考チュートリアル:

- [Python: Developing a License Plate Recognition System with Machine Learning in Python](#)
- [Python: Building a Facial Recognition Pipeline with Deep Learning in Tensorflow](#)

Day 153: 画像前処理パイプライン

Claude Codeへの依頼:

OpenCVで画像前処理パイプラインを作って。
リサイズ、グレースケール変換、ヒストグラム平坦化、
ノイズ除去、エッジ検出、二値化。

Day 154: 特徴量抽出

Claude Codeへの依頼:

HOG (Histogram of Oriented Gradients) 特徴量の実装。
SVM分類器と組み合わせた物体検出。

Day 155: 顔検出パイプライン

Claude Codeへの依頼:

OpenCVのHaarカスケード + dlibのランドマーク検出で
顔検出パイプラインを構築。
顔の位置、向き、表情の推定。

Day 156: ナンバープレート認識

Claude Codeへの依頼:

画像からナンバープレートを検出、
文字をセグメンテーションしてOCR認識するシステム。

Build Your Own 3D Renderer (Day 157-164)

参考チュートリアル:

- [C++: Introduction to Ray Tracing](#)
- [C++: How OpenGL works: software rendering in 500 lines of code](#)
- [C++: Raycasting engine of Wolfenstein 3D](#)

- [C++: Ray Tracing in One Weekend](#)
- [C++: Rasterization: a Practical Implementation](#)
- [C# / TypeScript / JavaScript: Learning how to write a 3D soft engine from scratch](#)
- [Java / JavaScript: Build your own 3D renderer](#)
- [Java: How to create your own simple 3D render engine in pure Java](#)
- [JavaScript / Pseudocode: Computer Graphics from scratch](#)
- [Python: A 3D Modeller](#)

Day 157: ベクトルと行列の数学ライブラリ

Claude Codeへの依頼:

3D数学ライブラリをPythonで作って。
Vec3, Mat4 クラス。内積、外積、正規化、
変換行列（平行移動、回転、拡大縮小、射影）。

Day 158: ワイヤーフレームレンダラ

Claude Codeへの依頼:

3Dワイヤーフレームレンダラ。
OBJファイル読み込み、透視投影、
Pillowで画像出力。カメラ回転のアニメーション。

Day 159: ラスタライゼーション

Claude Codeへの依頼:

三角形のラスタライゼーション。
バリセントリック座標、Zバッファ、
フラットシェーディング。

Day 160: テクスチャマッピングと照明

Claude Codeへの依頼:

UV座標によるテクスチャマッピング。
Phongの反射モデル（環境光、拡散光、鏡面反射）。

Day 161: レイトレーサー（基本）

Claude Codeへの依頼:

レイトレーサーをPythonで作って。
球体とのレイ交差判定、
ポイントライト、シャドウ。

Day 162: レイトレーサー（反射と屈折）

Claude Codeへの依頼:

反射（鏡面）と屈折（ガラス、水）を追加。
フレネル効果、複数バウンス。

Day 163: レイキャスティング（Wolfenstein 3D風）

Claude Codeへの依頼:

Wolfenstein 3D風のレイキャスティングエンジン。
2Dマップからの擬似3D表示。
HTML5 Canvas + JavaScriptで実装。

Day 164: Three.js統合プロジェクト

Claude Codeへの依頼:

Three.jsを使ったインタラクティブ3Dシーン。
自作レンダラ概念を使ったカスタムシェーダー。

Build Your Own Augmented Reality（Day 165-168）

参考チュートリアル:

- [C#: How To: Augmented Reality App Tutorial for Beginners with Vuforia and Unity 3D \[video\]](#)
- [C#: How To Unity ARCore \[video\]](#)
- [C#: AR Portal Tutorial with Unity \[video\]](#)
- [C#: How to create a Dragon in Augmented Reality in Unity ARCore \[video\]](#)
- [C#: How to Augmented Reality AR Tutorial: ARKit Portal to the Upside Down \[video\]](#)
- [Python: Augmented Reality with Python and OpenCV](#)

Day 165: ARマーカー検出

Claude Codeへの依頼:

OpenCVでARマーカー（ArUco）の検出。
カメラキャリブレーション、マーカーの姿勢推定。

Day 166: マーカー上に3Dオブジェクト描画

Claude Codeへの依頼:

検出したマーカーの上にOpenCVで3Dキューブを描画。
射影変換を使ったオーバーレイ。

Day 167: 平面検出と特徴点追跡

Claude Codeへの依頼:

ORB特徴量マッチングによる平面検出。
ホモグラフィ行列の計算と画像ワーピング。

Day 168: AR統合デモ

Claude Codeへの依頼:

Webカメラ映像にリアルタイムで3Dオブジェクトを
重ね合わせるARアプリケーション。

Phase 7: 応用・統合（Day 169-189）

Build Your Own Blockchain（Day 169-175）

参考チュートリアル:

- [Python: Learn Blockchains by Building One](#)
- [Python: Build your own blockchain: a Python tutorial](#)
- [Python: A Practical Introduction to Blockchain with Python](#)
- [Python: Let's Build the Tiniest Blockchain](#)
- [JavaScript: A cryptocurrency implementation in less than 1500 lines of code](#)
- [JavaScript: Build your own Blockchain in JavaScript](#)
- [JavaScript: Creating a blockchain with JavaScript](#)
- [TypeScript: Naivecoin: a tutorial for building a cryptocurrency](#)

- その他 C#, Crystal, Go, Java, Kotlin, Ruby, Scala, Rust, ATS のチュートリアルあり

Day 169: ブロックとチェーン

Day 170: Proof of Work

Day 171: トランザクション

Day 172: ウォレットと署名

Day 173: P2Pネットワーク

Day 174: コンセンサスアルゴリズム

Day 175: ブロックチェーンエクスプローラーUI

Build Your Own Emulator / Virtual Machine (Day 176-181)

参考チュートリアル:

- [C: Home-grown bytecode interpreters](#)
- [C: Virtual machine in C](#)
- [C: Write your Own Virtual Machine](#)
- [C: Writing a Game Boy emulator, Cinoop](#)
- [C++: How to write an emulator \(CHIP-8 interpreter\)](#)
- [C++: Emulation tutorial \(CHIP-8 interpreter\)](#)
- [C++: Emulation tutorial \(GameBoy emulator\)](#)
- [C++: NES Emulator From Scratch \[video\]](#)
- [Common Lisp: CHIP-8 in Common Lisp](#)
- [JavaScript: GameBoy Emulation in JavaScript](#)
- [Python: Emulation Basics: Write your own Chip 8 Emulator/Interpreter](#)
- [Rust: Odmg: Learning Rust by building a partial Game Boy emulator](#)

Day 176: CPU基礎とフェッチ-デコード-実行

Day 177: レジスタとメモリ

Day 178: 命令セット実装

Day 179: CHIP-8エミュレータ (前半)

Day 180: CHIP-8エミュレータ (後半)

Build Your Own Operating System 概念 & Uncategorized (Day 182-189)

OS参考チュートリアル:

- [C: Operating Systems: From 0 to 1](#)
- [C: The little book about OS development](#)
- [C: Roll your own toy UNIX-clone OS](#)
- [C: Kernel 101 - Let's write a Kernel](#)
- [C: How to create an OS from scratch](#)
- [C: Malloc tutorial](#)
- [C: Hack the virtual memory](#)
- [Rust: Writing an OS in Rust](#)
- [\(any\): Linux from scratch](#)
- その他多数のC, C++, Assembly チュートリアルあり

Uncategorized 参考チュートリアル:

- [\(any\): From NAND to Tetris](#)
- [\(any\): build-your-own-x-vibe-coding: BYOX-style tutorials adapted for vibe coding](#)
- [C: How to Write a Video Player in Less Than 1000 Lines](#)
- [C: Learn how to write a hash table in C](#)
- [C: The very basics of a terminal emulator](#)
- [C: Write a System Call](#)
- [C: Sol - An MQTT broker from scratch](#)
- [C++: How X Window Managers work and how to write one](#)
- [C++: Writing a Linux Debugger](#)
- [C#: C# Networking: Create a TCP chat server, TCP games, UDP Pong and more](#)
- [Clojure: Building a spell-checker](#)
- [Go: Build A Simple Terminal Emulator In 100 Lines of Golang](#)
- [Go: Let's Create a Simple Load Balancer](#)
- [Go: Video Encoding from Scratch](#)
- [JavaScript: Build Your Own Module Bundler - Minipack](#)
- [JavaScript: Learn JavaScript Promises by Building a Promise from Scratch](#)
- [JavaScript: Implementing promises from scratch \(TDD way\)](#)
- [JavaScript: JavaScript Algorithms and Data Structures](#)

- [JavaScript: Build a ride hailing app with React Native](#)
- [JavaScript: Build Your Own AdBlocker in \(Literally\) 10 Minutes](#)
- [Kotlin: Build Your Own Cache](#)
- [Lua: Building a CDN from Scratch to Learn about CDN](#)
- [Nim: Writing a Redis Protocol Parser](#)
- [Nim: Writing a Build system](#)
- [Nim: Writing a MiniTest Framework](#)
- [Nim: Writing a DMIDecode Parser](#)
- [Nim: Writing a INI Parser](#)
- [Nim: Writing a Link Checker](#)
- [Nim: Writing a URL Shortening Service](#)
- [Node.js: Build a static site generator in 40 lines with Node.js](#)
- [Node.js: Building A Simple Single Sign On\(SSO\) Server](#)
- [Node.js: Build a DNS Server in Node.js](#)
- [PHP: Write your own MVC from scratch in PHP](#)
- [PHP: Modern PHP Without a Framework](#)
- [PHP: Code a Web Search Engine in PHP](#)
- [Python: Build a Deep Learning Library \[video\]](#)
- [Python: How to Build a Kick-Ass Mobile Document Scanner in Just 5 Minutes](#)
- [Python: Continuous Integration System](#)
- [Python: Recommender Systems in Python: Beginner Tutorial](#)
- [Python: Write SMS-spam detector with Scikit-learn](#)
- [Python: A Simple Content-Based Recommendation Engine in Python](#)
- [Python: Stock Market Predictions with LSTM in Python](#)
- [Python: Building a simple Generative Adversarial Network \(GAN\) using Tensorflow](#)
- [Python: Learn ML Algorithms by coding: Decision Trees](#)
- [Python: JSON Decoding Algorithm](#)
- [Python: Build your own Git plugin with python](#)
- [Ruby: A Pedometer in the Real World](#)
- [Ruby: Creating a Linux Desktop application with Ruby](#)
- [Rust: Building a DNS server in Rust](#)
- [Rust: Writing Scalable Chat Service from Scratch](#)
- [Rust: WebGL + Rust: Basic Water Tutorial](#)

- TypeScript: Tiny Package Manager: Learns how npm or Yarn works

Day 182: メモリ管理（malloc実装）

Claude Codeへの依頼:

Pythonでmalloc/free のシミュレーション。
メモリプール、フリーリスト、
First Fit / Best Fit アルゴリズム。
フラグメンテーションの可視化。

Day 183: プロセススケジューラ

Claude Codeへの依頼:

プロセススケジューラのシミュレーション。
FIFO, Round Robin, Priority, CFS。
ガントチャートで可視化。

Day 184: ファイルシステム（概念実装）

Claude Codeへの依頼:

シンプルなファイルシステムのシミュレーション。
inode, ディレクトリエントリ、ブロック管理。
仮想ディスクイメージ上での操作。

Day 185: ハッシュテーブル完全実装

Claude Codeへの依頼:

ハッシュテーブルをスクラッチから実装。
衝突解決（チェイニング、オープンアドレッシング）、
動的リサイズ、ハッシュ関数の選択。

Day 186: ロードバランサー

Claude Codeへの依頼:

HTTPロードバランサーをPythonで作って。
Round Robin, Least Connections,
IP Hash の各アルゴリズム。ヘルスチェック機能。

Day 187: 静的サイトジェネレーター

Claude Codeへの依頼:

Day 188: CIシステム（継続的インテグレーション）

Claude Codeへの依頼:

Day 189: パッケージマネージャー

Claude Codeへの依頼:

Phase 8: 総合実践 & 卒業プロジェクト (Day 190-200)

Day 190-194: 卒業プロジェクト①「フルスタックWebアプリ」

Claude Codeに各パーツを生成してもらい、自分でつなぎ合わせ・デバッグ・改善する。

Day 195-198: 卒業プロジェクト②「AI搭載CLIツール」

Day 199: ポートフォリオ整理

Claude Codeへの依頼:

GitHubリポジトリの各プロジェクトのREADME.mdを

テンプレートに従って整備して。

バッジ、インストール手順、使い方、スクリーンショット、

ライセンスを含む。

Day 200: 振り返りと次のステップ

- 全200日の学習ログをまとめる
- 各カテゴリの理解度を自己評価
- Claude Code活用のベストプラクティスを文書化
- 今後の学習ロードマップを作成

 カテゴリ別カバレッジ一覧

#	カテゴリ	対応Day	主要言語
1	3D Renderer	157-164	Python, JS
2	Augmented Reality	165-168	Python
3	BitTorrent Client	98-103	Python
4	Blockchain / Cryptocurrency	169-175	Python
5	Bot	104-112	Python
6	Command-Line Tool	24-28	Python
7	Database	39-48	Python
8	Docker	92-97	Python
9	Emulator / Virtual Machine	176-181	Python
10	Front-end Framework / Library	113-122	JavaScript

#	カテゴリ	対応Day	主要言語
11	Game	123-133	JS, Python
12	Git	73-79	Python
13	Network Stack	85-91	Python
14	Neural Network	141-152	Python
15	Operating System	182-184	Python (概念)
16	Physics Engine	134-137	JS, Python
17	Programming Language	57-72	Python
18	Regex Engine	15-19	Python
19	Search Engine	49-53	Python
20	Shell	8-14	Python
21	Template Engine	20-23	Python
22	Text Editor	80-84	Python
23	Visual Recognition System	153-156	Python
24	Voxel Engine	138-140	Python, JS
25	Web Browser	54-56	Python
26	Web Server	29-38	Python
27	Uncategorized	182-189	Python, JS

 Claude Code 効率的な使い方 Tips

プロンプトテンプレート

新機能を作る時:

[技術名]を[言語]で実装して。

要件:

- [機能1]
- [機能2]
- [機能3]

テスト付きで、エラーハンドリングも含めて。

バグ修正を依頼する時（学習段階では自分でやるのが最優先）:

以下のエラーが出ています。原因を説明して、
修正方法を3つ提案して。ただしコードは書かないで。
[エラーメッセージ]

リファクタリングを依頼する時:

このコードをリファクタリングして。

- SOLID原則に従う
- 関数は20行以内
- docstringを追加
- 型ヒントを追加

デバッグの心得

1. エラーメッセージを最後の行から読む（Pythonのトレースバック）
2. まず自分で5分考える→ダメならClaude Codeにヒントだけ聞く
3. **print**文デバッグ→**VSCode**ブレークポイント→**logging** の順で試す
4. 同じエラーは2回目から自分で解決する（エラーノートを作る）

注意: 一部の参考チュートリアルURLは時間の経過により閲覧不能になっている場合があります。その場合は該当チュートリアルのテーマ名でWeb検索し、代替リソースを探すか、Claude Codeにそのテーマの実装を直接依頼してください。

カリキュラム作成元: codecrafters-io/build-your-own-x (CC0 ライセンス) 全28カテゴリ・約180+ チュートリアルを網羅